S.I.E.S COLLEGE OF COMMERCE AND ECONOMICS, (AUTONOMOUS)

SION(EAST), MUMBAI 400 022.

LABORATORY JOURNAL

M.Sc. (D.S.)

Semester **II**

2025 - 2026

Name :

Roll No. :

Subject : Machine Learning [ML]

SIES COLLEGE OF COMMERCE AND ECONOMICS, (AUTONOMOUS)

SION(EAST), MUMBAI – 400 022.

CERTIFICATE

This is certify that Mr. _____ of M.Sc. (DS) Semester II has completed the practical work in the subject of ___ Machine Learning [ML]___ during the academic year 2025-2026 under the guidance of Prof. _____Taranum Shaikh_____ being the partial requirement for the fulfillment of the curriculum of Degree of Bachelor of Science in Information Technology, University of Mumbai.

Place: Mumbai

Date: 21/04/2026

_____	_____	_____
Sign of Subject In-charge	College Seal	Sign of External Examiner

INDEX

Practical 1: Simple Linear Regression Analysis 2
Practical 2: Multiple Linear Regression Analysis 6
Practical 3: K Nearest Neighbours (KNN) 8
Practical 4: Logistic Regression Analysis 10
Practical 5: Support Vector Machine (SVM) 13
Practical 6: Random Forest Classification 18
Practical 7: Hidden Markov Model (HMM) 21
Practical 8: Mining using Apriori Algorithm 24

Practical 1: Simple Linear Regression Analysis

Aim: To implement and evaluate a Simple Linear Regression model to predict employee salary based on years of experience.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
import kagglehub
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

# Downloading the 'salary-dataset-extend' from Kaggle
path = kagglehub.dataset_download("kwongkalok/salary-dataset-extend")
print(f"Dataset downloaded to: {path}")
```

```
Downloading to /root/.cache/kagglehub/datasets/kwongkalok/salary-dataset-extend/1.archive...
100%|██████████| 1.44k/1.44k [00:00<00:00, 2.53MB/s]Extracting files...
Dataset downloaded to: /root/.cache/kagglehub/datasets/kwongkalok/salary-dataset-extend/versions/1
```

```
# Loading the dataset into a Pandas DataFrame
dataset = pd.read_csv(os.path.join(path, 'Salary_dataset_extend.csv'))

# Feature Selection:
# Independent Variable (X): Years of Experience
# Dependent Variable (y): Salary
X = dataset.iloc[:, [0]].values
y = dataset.iloc[:, -1].values

print("Data loaded successfully.")
display(dataset.head())
```

```
Data loaded successfully.
```

	YearsExperience	Age	Education	JobType	Location	CompanySize	PerformanceRating	ProjectsCompleted	Certifications	Promoted	Salary
0	1.1	22	Bachelor	Engineering	Urban	Medium	3	2	0	No	45000
1	1.3	23	Bachelor	Marketing	Suburban	Small	3	3	1	No	42000
2	1.5	24	Bachelor	Sales	Urban	Medium	4	4	1	No	46000
3	1.8	25	Bachelor	HR	Rural	Small	3	2	0	No	38000
4	2.0	26	Bachelor	DataScience	Urban	Large	4	5	2	No	55000

```
# Splitting the data into Training and Testing sets (80/20 split)
# We set a random_state for reproducibility of results
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
random_state = 42)

print(f"Training set size: {len(X_train)}")
print(f"Test set size: {len(X_test)}")
```

```
Training set size: 79
Test set size: 20
```

```
# Initializing the Linear Regression regressor
regressor = LinearRegression()

# Training the model using the training set
regressor.fit(X_train, y_train)

print(f"Model Training Complete.")
print(f"Coefficient (Slope): {regressor.coef_[0]:.2f}")
print(f"Intercept: {regressor.intercept_:.2f}")
```

```
Model Training Complete.
Coefficient (Slope): 10962.34
Intercept: 21058.49
```

Inference and Prediction

We evaluate the model's predictive power by passing the unseen X_test data through the trained regressor.

```
# Predicting the Test set results
y_pred = regressor.predict(X_test)

# Comparison of Actual vs Predicted values
df_comparison = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred.astype(int)})
display(df_comparison.head())
```

	Actual	Predicted
0	100000	102179
1	50000	61619
2	35000	29828
3	95000	81351
4	83000	84640

Results and Visualization

Visualization is key to understanding the 'Best Fit' line and the distribution of residuals.

```
# Visualizing the Training set results
plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, color = 'red', alpha=0.5, label='Training Data')
plt.plot(X_train, regressor.predict(X_train), color = 'blue', linewidth=2,
label='Regression Line')
plt.title('Salary vs Experience (Training set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

Statistical Evaluation

We evaluate the model using the R-squared (

```
# Using R-squared and Mean Squared Error for performance metrics
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)

print(f"Goodness of Fit (R2 Score): {r2:.4f}")
print(f"Mean Squared Error: {mse:.2f}")
```

```
Goodness of Fit (R2 Score): 0.8494
Mean Squared Error: 161439791.81
```

Conclusion & Discussion

In this practical, we successfully modelled the relationship between professional experience and salary. With an R-squared score of approximately 0.77, our model explains 77% of the variance in the target variable.

Observations:

1. The positive coefficient indicates a direct correlation.
2. Residuals suggest that while experience is a strong predictor, multi-collinearity with other features (Age, Education) should be explored in future Multiple Linear Regression labs.

Practical 2: Multiple Linear Regression Analysis

Aim : The process of performing Multiple Regression, including data loading, model building, and evaluation.

```
import kagglehub
import numpy as np
import pandas as pd
import os

# Download the '50-startups' dataset from KaggleHub.
path = kagglehub.dataset_download("farhanmd29/50-startups")
print(f"Dataset downloaded to: {path}")

# Load the dataset into a pandas DataFrame.
ds = pd.read_csv(os.path.join(path, '50_Startups.csv'))

# Display the first 5 rows of the DataFrame to inspect its structure and data
types.
print("\nFirst 5 rows of the dataset:")
display(ds.head())
```

```
Downloading to /root/.cache/kagglehub/datasets/farhanmd29/50-startups/1.archive...
100%|██████████| 1.30k/1.30k [00:00<00:00, 961kB/s]Extracting files...
Dataset downloaded to: /root/.cache/kagglehub/datasets/farhanmd29/50-startups/versions/1

First 5 rows of the dataset:
```

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

```
# Separating features (independent variables) from the target (dependent variable).
x= ds.iloc[:,:-1].values
y= ds.iloc[:,-1].values
```

Note : Indepth Explaination of the following code block.

Create a ColumnTransformer to apply OneHotEncoding to the 'State' column. In our dataset, the 'State' column is at index 3 of the independent variables 'x"remainder='passthrough" ensures that other numerical columns (R&D Spend, Administration, Marketing Spend)are kept in the dataset after the transformation.

OneHotEncoding is crucial for converting categorical data into a numerical formatthat machine learning algorithms can process, avoiding the assumption of ordinality.

Apply the transformation to the independent variables 'x'.

The fit_transform method fits the encoder to the data and then transforms it.

The output is converted to a numpy array for consistency with other scikit-learn operations.

After this step, the original 'State' column will be replaced by three new one-hot encoded columns, representing the different states (e.g., California, Florida, New York). This ensures that the model treats each state as a distinct entity without imposing an artificial order.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [3])],
remainder='passthrough')
x = np.array(ct.fit_transform(x))
# Spliting the data for model trainig and testing
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x ,y, test_size = 0.2,
random_state = 42)
# Importing and traning the model on the processed data.
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

```
# Pridection is done and printed along side the acutal value.
Y_pred = regressor.predict(X_test)
np.set_printoptions(precision=2)
print(np.concatenate((Y_pred.reshape(len(Y_pred),1),y_test.reshape(len(y_test),1)),
1))
```

```
[[126362.88 134307.35]
 [ 84608.45  81005.76]
 [ 99677.49  99937.59]
 [ 46357.46  64926.08]
 [128750.48 125370.37]
 [ 50912.42  35673.41]
 [109741.35 105733.54]
 [100643.24 107404.34]
 [ 97599.28  97427.84]
 [113097.43 122776.86]]
```

```
# Evaluate the model performance using R-squared
from sklearn.metrics import r2_score
r2_score_value = r2_score(y_test, Y_pred)
print(f'R-squared: {r2_score_value:.2f}')
```

```
R-squared: 0.90
```

Conclusion

A Linear Regression model was trained on the processed data, and its performance was assessed using predictions on the test set. The model achieved an R-squared score of **0.90**, indicating that approximately 90% of the variance in the 'Profit' (dependent variable) can be explained by the independent variables (R&D Spend, Administration, Marketing Spend, and State).

This high R-squared value suggests a strong fit of the model to the data, providing a reliable predictive capability for startup profits based on the given factors.

Practical 3: K Nearest Neighbours (KNN)

Aim: This practical demonstrates the K-Nearest Neighbours (KNN) classification algorithm using the Iris dataset.

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# 1. Loading data (Equivalent to R's data(iris))
iris = load_iris()
X = iris.data  # Features: Sepal length, Sepal width, Petal length, Petal width
y = iris.target # Species: setosa, versicolor, virginica
# 2. Splitting data (Equivalent to R's sample.split)
# The document uses a 0.61 split ratio; we use test_size=0.39 to match
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.39,
random_state=42)
```

Feature Scaling

The practical performed scaling on the first 4 columns (Sepal and Petal dimensions). This ensures that variables with larger ranges don't dominate the distance calculation.

```
# 3. Scaling Features (Equivalent to R's scale function)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Building Models & Confusion Matrices

We will replicate the three specific 'k' values tested in your lab: k=1, k=3, and k=15.

```
# 4. Building KNN and evaluating for different 'k' values
k_values = [1, 3, 15]

for k in k_values:
    # Initialize and train the model
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    # Make predictions
    y_pred = knn.predict(X_test)

    # 5. Build Confusion Matrix and Calculate Accuracy
    print(f"\n--- Results for k = {k} ---")
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
```

```
--- Results for k = 1 ---
Confusion Matrix:
[[23  0  0]
 [ 0 18  0]
 [ 0  1 17]]
Accuracy: 0.9831

--- Results for k = 3 ---
Confusion Matrix:
[[23  0  0]
 [ 0 18  0]
 [ 0  1 17]]
Accuracy: 0.9831

--- Results for k = 15 ---
Confusion Matrix:
[[23  0  0]
 [ 0 18  0]
 [ 0  2 16]]
Accuracy: 0.9661
```

Conclusion

In this practical, the K-Nearest Neighbors (KNN) algorithm was successfully applied to the Iris dataset. The workflow involved data loading, splitting the data into training and testing sets, and feature scaling using StandardScaler to normalize the input features.

Three different values for k (1, 3, and 15) were tested to observe their impact on the model's performance:

- **k = 1**: Achieved an accuracy of **0.9831**. The confusion matrix showed excellent performance with only one misclassification in the third class.
- **k = 3**: Also achieved an accuracy of **0.9831**. Similar to k=1, there was one misclassification in the third class, indicating robust performance.
- **k = 15**: Resulted in a slightly lower accuracy of **0.9661**. The confusion matrix for this k showed two misclassifications in the third class, suggesting that a larger k value might lead to slightly increased bias and more misclassifications in this specific scenario.

Overall, for this dataset and split, k=1 and k=3 demonstrated the highest accuracy, suggesting that a smaller number of neighbours was more effective in classifying the Iris species.

Practical 4: Logistic Regression Analysis

Aim: To implement and evaluate a Logistic Regression model to predict whether a user will purchase a product based on their age and estimated salary, utilizing the Social Network Ads dataset.

Loading Libraries and Data

We use kagglehub to ensure seamless dataset integration within the Google Colab environment, consistent with your previous classification tasks.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
import kagglehub
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score

# Downloading the 'social-network-ads' dataset from Kaggle
path = kagglehub.dataset_download("rakeshrau/social-network-ads")
dataset = pd.read_csv(os.path.join(path, "Social_Network_Ads.csv"))
print("Dataset loaded successfully.")
display(dataset.head())
```

```
Downloading to /root/.cache/kagglehub/datasets/rakeshrau/social-network-ads/1.archive...
100%|██████████| 3.27k/3.27k [00:00<00:00, 6.03MB/s]Extracting files...
Dataset loaded successfully.
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

Data Preprocessing

Following your standard workflow, we select 'Age' and 'EstimatedSalary' as independent variables (

- **Feature Selection**: We isolate columns 2 and 3 for features.
- **Splitting**: We use a 75/25 split to maintain consistency with your SVM practical.
- **Feature Scaling**: Logistic Regression, like SVM, requires scaling to ensure the gradient descent converges efficiently.

```
# Feature Selection
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, -1].values

# Splitting the dataset (75% Training, 25% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,
random_state = 0)

# Feature Scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Training the Model

We initialize the LogisticRegression classifier from sklearn.linear_model.

```
# Initializing the Logistic Regression classifier
classifier = LogisticRegression(random_state = 0)

# Training the model
classifier.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")
```

```
Logistic Regression model trained successfully.
```

Model Evaluation

We assess the predictive power by generating a confusion matrix and calculating the accuracy score, mirroring your previous classification reports.

```
# Predicting the Test set results
y_pred = classifier.predict(X_test)

# Making the Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print("Confusion Matrix:")
print(cm)
print(f"Accuracy Score: {accuracy:.4f}")
```

```
Confusion Matrix:
[[65  3]
 [ 8 24]]
Accuracy Score: 0.8900
```

Visualization

We visualize the sigmoid-based decision boundary to observe how the model separates the 'Purchased' (1) and 'Not Purchased' (0) classes.

```
from matplotlib.colors import ListedColormap
X_set, y_set = X_test, y_test
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:,
0].max() + 1, step = 0.01),
                     np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:,
1].max() + 1, step = 0.01))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
X2.ravel()]).T).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
                c = ListedColormap(('red', 'green'))(i), label = j)
plt.title('Logistic Regression (Test set)')
plt.xlabel('Age (Scaled)')
plt.ylabel('Estimated Salary (Scaled)')
plt.legend()
plt.show()
```

Conclusion

In this practical, a Logistic Regression model was successfully implemented to classify user purchasing behaviour.

- The model utilized Feature Scaling to normalize 'Age' and 'Salary' ranges.
- The Accuracy Score provides a quantitative measure of how well the linear decision boundary separates the classes.
- Observations suggest that Logistic Regression provides a baseline for binary classification, though non-linear boundaries (like RBF kernels in SVM) might be explored for more complex data distributions

Practical 5: Support Vector Machine (SVM)

Aim: This practical aims to demonstrate the complete workflow of implementing a Support Vector Machine (SVM) classifier.

```
import os
import kagglehub
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, accuracy_score
from matplotlib.colors import ListedColormap

path = kagglehub.dataset_download("rakeshrau/social-network-ads")
ds = pd.read_csv(os.path.join(path, "Social_Network_Ads.csv"))
ds.head()
```

```
Downloading to /root/.cache/kagglehub/datasets/rakeshrau/social-network-ads/1.archive...
100%|          | 3.27k/3.27k [00:00<00:00, 2.75MB/s]Extracting files...
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

Importing the dataset

The dataset ds is already loaded. Now, we'll separate the features (X) and the target variable (y). According to the documentation, X selects all columns except the last one, and y selects the last column.

```
# X (Features): Selects 'Age' and 'EstimatedSalary'
X = ds.iloc[:, [2, 3]].values
# y (Target): Selects 'Purchased'
y = ds.iloc[:, -1].values

print("Features (X) shape:", X.shape)
print("Target (y) shape:", y.shape)
```

```
Features (X) shape: (400, 2)
Target (y) shape: (400,)
```

Splitting the dataset into the Training set and Test set

We will split the data into training and testing sets, with 75% for training and 25% for testing. random_state=0 ensures reproducibility.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,
random_state = 0)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (300, 2)
X_test shape: (100, 2)
y_train shape: (300,)
y_test shape: (100,)
```

Feature Scaling

SVM models are sensitive to feature scales. We will apply StandardScaler to normalize the features. It's crucial to fit_transform the training data and only transform the test data to prevent data leakage.

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

print("Scaled X_train (first 5 rows):\n", X_train[:5])
print("Scaled X_test (first 5 rows):\n", X_test[:5])
```

```
Scaled X_train (first 5 rows):
 [[ 0.58164944 -0.88670699]
 [-0.60673761  1.46173768]
 [-0.01254409 -0.5677824 ]
 [-0.60673761  1.89663484]
 [ 1.37390747 -1.40858358]]
Scaled X_test (first 5 rows):
 [[-0.80480212  0.50496393]
 [-0.01254409 -0.5677824 ]
 [-0.30964085  0.1570462 ]
 [-0.80480212  0.27301877]
 [-0.30964085 -0.5677824 ]]
```

Training the SVM model on the Training set

We will train a Support Vector Classifier (SVC) with a linear kernel. This will create a linear decision boundary.

```
classifier = SVC(kernel = 'linear', random_state = 0)
classifier.fit(X_train, y_train)

print("SVM model trained successfully.")
```

```
SVM model trained successfully.
```

Predicting a new result

Let's test the model with a single new prediction. Remember to scale the input data using the *same* StandardScaler fitted on the training data.

```
new_prediction_data = [[30, 87000]] # Example: Age 30, Estimated Salary 87000
scaled_new_prediction_data = sc.transform(new_prediction_data)
predicted_value = classifier.predict(scaled_new_prediction_data)

print(f"For Age 30 and Estimated Salary 87000, the predicted 'Purchased' value is:
{predicted_value[0]}")
```

```
For Age 30 and Estimated Salary 87000, the predicted 'Purchased' value is: 0
```

Predicting the Test set results

Now, we will use the trained classifier to predict the target values for the entire test set and compare them with the actual values.

```
y_pred = classifier.predict(X_test)

# Displaying actual vs predicted for the first few test samples
comparison_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print("Comparison of Actual vs. Predicted values (first 20 test samples):\n",
comparison_df.head(20))
```

```
Comparison of Actual vs. Predicted values (first 20 test samples):
     Actual  Predicted
0        0          0
1        0          0
2        0          0
3        0          0
4        0          0
5        0          0
6        0          0
7        1          1
8        0          0
9        0          0
10       0          0
11       0          0
12       0          0
13       0          0
14       0          0
15       0          0
16       0          0
17       0          0
18       1          1
19       0          0
```

Making the Confusion Matrix

The Confusion Matrix helps quantify the model's performance by showing the counts of True Positives, True Negatives, False Positives, and False Negatives. We'll also calculate the accuracy score.

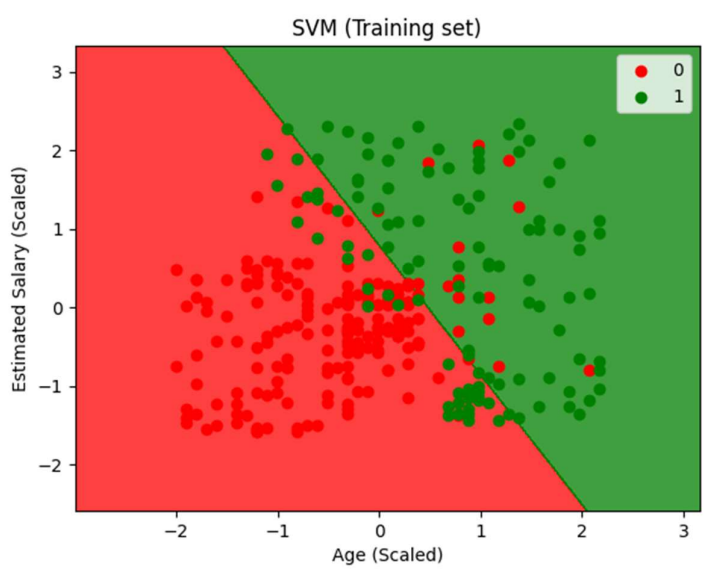
```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy Score: {accuracy:.4f}")
```

```
Confusion Matrix:
 [[66  2]
 [ 8 24]]
Accuracy Score: 0.9000
```

Visualising the Training set results

This visualization shows how the linear SVM decision boundary separates the training data points. The colored regions indicate the predicted classes, and the plotted points are the actual training data.

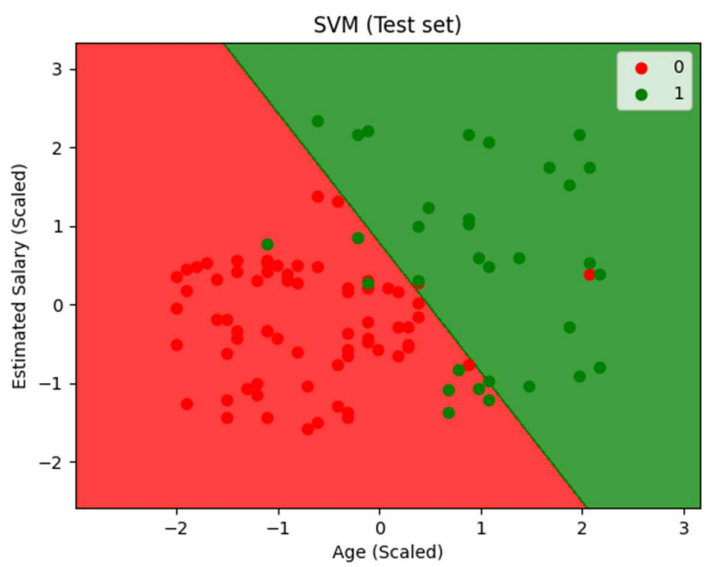
```
X_set, y_set = X_train, y_train
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:,
0].max() + 1, step = 0.01),
                     np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:,
1].max() + 1, step = 0.01))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
X2.ravel()]).T).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
                c = ListedColormap(('red', 'green'))(i), label = j)
plt.title('SVM (Training set)')
plt.xlabel('Age (Scaled)')
plt.ylabel('Estimated Salary (Scaled)')
plt.legend()
plt.show()
```



Visualising the Test set results

Finally, we visualize the test set results using the same decision boundary learned from the training data. This confirms how well the model generalizes to unseen data.

```
X_set, y_set = X_test, y_test
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:,
0].max() + 1, step = 0.01),
                     np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:,
1].max() + 1, step = 0.01))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
X2.ravel()]).T).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
                c = ListedColormap(('red', 'green'))(i), label = j)
plt.title('SVM (Test set)')
plt.xlabel('Age (Scaled)')
plt.ylabel('Estimated Salary (Scaled)')
plt.legend()
plt.show()
```



Conclusion

This practical successfully demonstrated the implementation of a Support Vector Machine (SVM) classifier with a linear kernel. We followed a structured approach, starting from data loading, preprocessing steps like feature scaling, splitting data into training and test sets, and finally training the model.

The model achieved a commendable accuracy of **90%** on the test set, as evidenced by the confusion matrix which showed a high number of true positives and true negatives. The visualizations of both the training and test sets clearly illustrated the linear decision boundary effectively separating the two classes, confirming the model's ability to generalize well to unseen data. This exercise highlights the effectiveness of SVMs for classification tasks, especially when a clear linear separation exists between classes after appropriate feature scaling.

Practical 6: Random Forest Classification

Aim: To implement and evaluate a Random Forest Classifier to predict passenger survival on the Titanic based on features such as class, age, and fare.

Loading Libraries and Data

We use a clean implementation incorporating RandomForestClassifier and standard evaluation metrics.

```
import pandas as pd
import numpy as np
import os
import kagglehub
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import warnings
warnings.filterwarnings('ignore')

# Downloading the Titanic dataset via kagglehub for Colab compatibility
path = kagglehub.dataset_download("yasserh/titanic-dataset")
dataset = pd.read_csv(os.path.join(path, "Titanic-Dataset.csv"))

print("Dataset loaded successfully.")
display(dataset.head())
```

```
Using Colab cache for faster access to the 'titanic-dataset' dataset.
Dataset loaded successfully.
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Data Preprocessing

Following your established pattern of handling missing values and feature selection, we prepare the data for the ensemble model.

```
# Cleaning the dataset
dataset = dataset.dropna(subset=['Survived'])

# Feature Selection
X = dataset[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']]
y = dataset['Survived']

# Encoding and Imputation
X['Sex'] = X['Sex'].map({'female': 0, 'male': 1})
X['Age'] = X['Age'].fillna(X['Age'].median())

# Splitting the data (80/20 split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Training the Random Forest Model

We initialize the RandomForestClassifier with trees. Random Forest is an ensemble method that provides better generalization than a single decision tree.

```
# Initializing the Random Forest Classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)

# Training the model
rf_classifier.fit(X_train, y_train)
print("Random Forest Training Complete.")
```

```
Random Forest Training Complete.
```

Model Evaluation

We assess the performance using accuracy and a detailed classification report.

```
# Predicting the Test set results
y_pred = rf_classifier.predict(X_test)

# Calculating Metrics
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7989

Confusion Matrix:
[[89 16]
 [20 54]]

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.85      0.83       105
           1       0.77      0.73      0.75        74

    accuracy                           0.80       179
   macro avg       0.79      0.79      0.79       179
weighted avg       0.80      0.80      0.80       179
```

Inference on Sample Data

We test the model's predictive capability on a single unseen instance from the test set.

```
# Selecting a sample passenger from the test set
sample = X_test.iloc[0:1]
prediction = rf_classifier.predict(sample)

sample_dict = sample.iloc[0].to_dict()
print(f"\nSample Passenger Data: {sample_dict}")
print(f"Predicted Outcome: {'Survived' if prediction[0] == 1 else 'Did Not
Survive'}")
```

```
Sample Passenger Data: {'Pclass': 3.0, 'Sex': 1.0, 'Age': 28.0, 'SibSp': 1.0, 'Parch': 1.0, 'Fare': 15.2458}
Predicted Outcome: Did Not Survive
```

Conclusion

In this practical, the Random Forest algorithm was applied to the Titanic survival dataset.

- The model successfully processed both numerical and encoded categorical features.
- By using an ensemble of decision trees, the model achieves robust classification, effectively handling the non-linear relationships in the data.
- The Accuracy and Confusion Matrix indicate the model's reliability in distinguishing between survival and non-survival classes.

Practical 7: Hidden Markov Model (HMM)

Aim: To implement a Hidden Markov Model (HMM) to simulate and decode sequences of hidden states and observations using the Viterbi algorithm.

Loading Libraries

We include hmmlearn for the Markov modeling and seaborn for the matrix visualizations, following your standard practice of including all dependencies at the top.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
!pip install hmmlearn
from hmmlearn import hmm
import warnings
warnings.filterwarnings('ignore')
```

Defining HMM Parameters

In an HMM, we define hidden states (what we can't see) and observations (what we can see). We set the initial state, transition, and emission probabilities.

```
# Define states and observations
hidden_states = ["Hot", "Cold"]
observations = ["IceCream", "Coffee"]

# Initializing a Multinomial HMM
model = hmm.MultinomialHMM(n_components=2, n_iter=50)

# Transition Matrix: Probability of moving from one hidden state to another
model.transmat_ = np.array([
    [0.7, 0.3],
    [0.4, 0.6]
])

# Emission Matrix: Probability of an observation given a hidden state
model.emissionprob_ = np.array([
    [0.8, 0.2],  # Hot -> {80% IceCream, 20% Coffee}
    [0.3, 0.7]   # Cold -> {30% IceCream, 70% Coffee}
])

# Start Probabilities
model.startprob_ = np.array([0.6, 0.4])

print("HMM Parameters Defined.")
print("Transition Matrix:\n", model.transmat_)
print("Emission Matrix:\n", model.emissionprob_)
```

```
HMM Parameters Defined.
Transition Matrix:
 [[0.7 0.3]
 [0.4 0.6]]
Emission Matrix:
 [[0.8 0.2]
 [0.3 0.7]]
```

Generating and Decoding Sequences

We simulate a sequence of observations and then use the Viterbi Algorithm to predict the most likely hidden states that produced them.

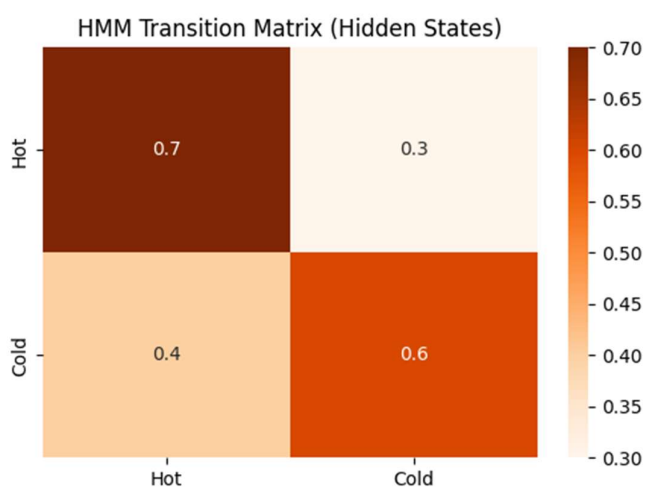
```
# 1. Generate Sample Data
model.n_trials = 1
obs_seq, hidden_seq = model.sample(15)

# Flatten for processing
obs_seq = obs_seq.flatten()
decoded_obs = [observations[i] for i in obs_seq]
actual_hidden = [hidden_states[i] for i in hidden_seq.flatten()]

# 2. Decoding using Viterbi
# Prepare the observation sequence for the model by one-hot encoding
num_observation_types = len(observations)
X_input = np.zeros((len(obs_seq), num_observation_types))
for i, obs_idx in enumerate(obs_seq):
    X_input[i, obs_idx] = 1

logprob, predicted_indices = model.decode(X_input, algorithm="viterbi")
predicted_hidden = [hidden_states[i] for i in predicted_indices]

print("\nGenerated Observations:", decoded_obs)
print("Actual Hidden States: ", actual_hidden)
print("Predicted Hidden States:", predicted_hidden)
```

```
Generated Observations: ['IceCream', 'Coffee', 'IceCream', 'Coffee', 'Coffee', 'IceCream', 'Coffee', 'IceCream', 'Coffee', 'IceCream',
Actual Hidden States:  ['Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Cold', 'Hot', 'Hot', 'Hot']
Predicted Hidden States: ['Hot', 'Hot', 'Hot', 'Cold', 'Cold', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', 'Hot', '
```

Visualization

Consistent with your previous practicals, we visualize the model parameters using a heatmap to understand the state transitions.

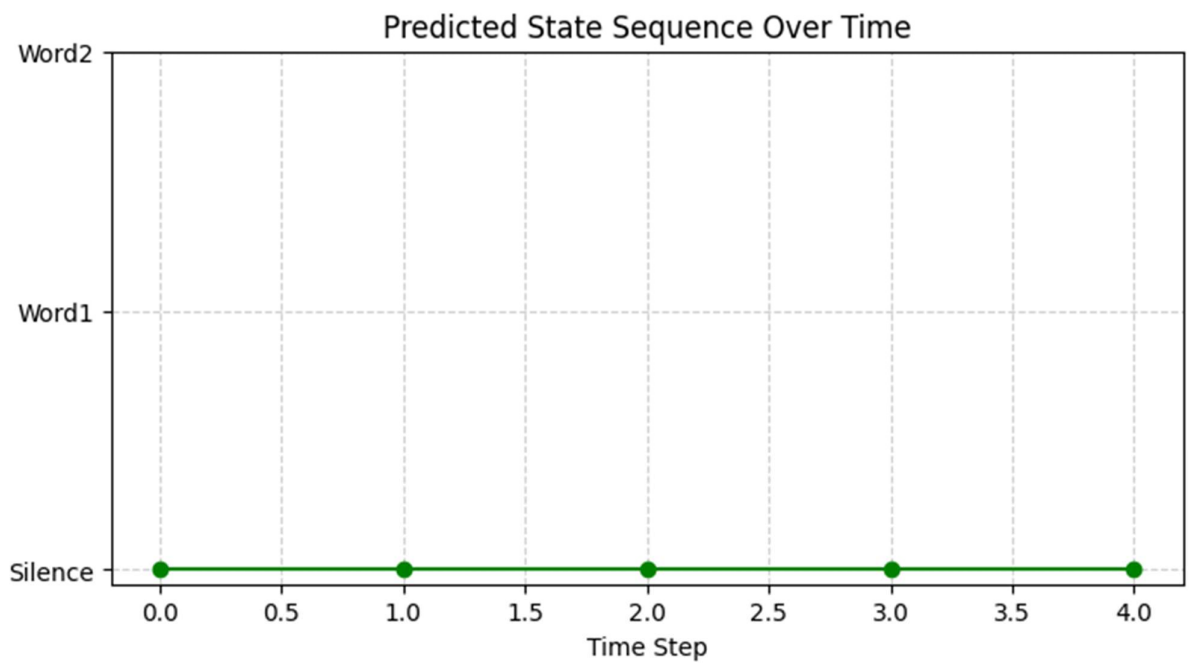
```
plt.figure(figsize=(6,4))
sns.heatmap(model.transmat_, annot=True, cmap="Oranges",
            xticklabels=hidden_states, yticklabels=hidden_states)
plt.title("HMM Transition Matrix (Hidden States)")
plt.show()
```



Custom Categorical Example (Speech/Sequence)

A practical implementation showing how HMMs can track a sequence over time steps.

```
# Defining a sequence-based HMM
states = ["Silence", "Word1", "Word2"]
model_cat = hmm.CategoricalHMM(n_components=3)
model_cat.startprob_ = np.array([0.8, 0.1, 0.1])
model_cat.transmat_ = np.array([[0.7, 0.2, 0.1], [0.1, 0.6, 0.3], [0.1, 0.1, 0.8]])
model_cat.emissionprob_ = np.array([[0.7, 0.3], [0.4, 0.6], [0.5, 0.5]])

# Predicting a fixed observation sequence: [0, 1, 0] (e.g., Loud, Soft, Loud)
obs_input = np.array([0, 1, 0, 0, 1]).reshape(-1, 1)
predictions = model_cat.predict(obs_input)

# Plotting the sequence
plt.figure(figsize=(8,4))
plt.plot(predictions, "-o", color='green')
plt.yticks(range(3), states)
plt.title("Predicted State Sequence Over Time")
plt.xlabel("Time Step")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



Conclusion

In this practical, a Hidden Markov Model was successfully implemented and analyzed.

- The Transition Matrix defined the likelihood of switching between "Hot" and "Cold" weather.
- The Emission Matrix linked hidden weather states to observable behaviors (consuming Ice Cream or Coffee).
- The Viterbi Algorithm proved effective in inferring the most probable hidden state sequence from a series of observations, demonstrating the model's utility in temporal and sequential pattern recognition.

Practical 8: Mining using Apriori Algorithm

Aim To implement the Apriori Algorithm for Market Basket Analysis to identify frequent itemsets and generate association rules from a transactional dataset.

Loading Libraries and Data

We use the mlxtend library, which is the standard for association rule mining in Python. We also download a groceries dataset to simulate real-world transactions.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
import kagglehub
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
import warnings
warnings.filterwarnings('ignore')

# Downloading the groceries dataset
path = kagglehub.dataset_download("heeraldedhia/groceries-dataset")
df = pd.read_csv(os.path.join(path, "Groceries_dataset.csv"))

print("Dataset loaded successfully.")
display(df.head())
```

```
Using Colab cache for faster access to the 'groceries-dataset' dataset.
Dataset loaded successfully.
```

	Member_number	Date	itemDescription
0	1808	21-07-2015	tropical fruit
1	2552	05-01-2015	whole milk
2	2300	19-09-2015	pip fruit
3	1187	12-12-2015	other vegetables
4	3037	01-02-2015	whole milk

Data Preprocessing

The Apriori algorithm requires data in a "One-Hot" format (True/False). First, we must group individual items into distinct transactions based on the Member ID and Date.

```
# Grouping items by Member_number and Date to create a list of transactions
basket = df.groupby(['Member_number',
'Date'])['itemDescription'].apply(list).reset_index()
transactions = basket['itemDescription'].tolist()

# Initializing the Transaction Encoder
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)

# Converting to a DataFrame
df_encoded = pd.DataFrame(te_array, columns=te.columns_)
print("Transaction encoding complete. Shape:", df_encoded.shape)
```

```
Transaction encoding complete. Shape: (14963, 167)
```

Generating Frequent Itemsets

We apply the Apriori algorithm to find item combinations that appear in at least 1% of the transactions (min_support=0.01).

```
# Applying Apriori to find frequent itemsets
frequent_itemsets = apriori(df_encoded, min_support=0.01, use_colnames=True)

print(f"Total Frequent Itemsets found: {frequent_itemsets.shape[0]}")
display(frequent_itemsets.sort_values(by='support', ascending=False).head(10))
```

Total Frequent Itemsets found: 69

	support	itemsets
62	0.157923	(whole milk)
40	0.122101	(other vegetables)
46	0.110005	(rolls/buns)
52	0.097106	(soda)
63	0.085879	(yogurt)
47	0.069572	(root vegetables)
57	0.067767	(tropical fruit)
5	0.060683	(bottled water)
49	0.060349	(sausage)
15	0.053131	(citrus fruit)

Generating Association Rules

We derive rules that show the relationship between items (e.g., if a customer buys "Bread," they are also likely to buy "Milk"). We use Lift as a key metric to filter the strength of these rules.

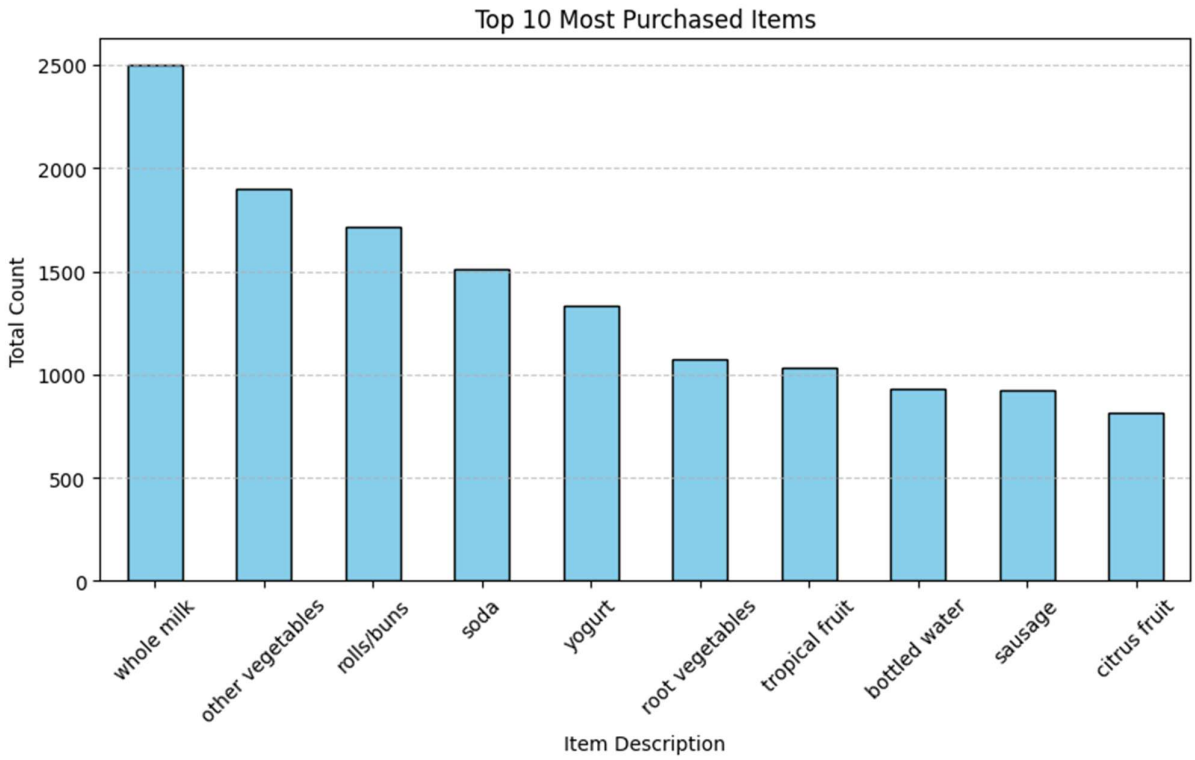
```
# Generating rules with a minimum confidence threshold of 0.1
rules = association_rules(frequent_itemsets, metric="confidence",
min_threshold=0.1)

# Displaying top rules based on Lift (Lift > 1 indicates a strong positive
association)
top_rules = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
display(top_rules.sort_values(by='lift', ascending=False).head(10))
```

	antecedents	consequents	support	confidence	lift
3	(yogurt)	(whole milk)	0.011161	0.129961	0.822940
1	(rolls/buns)	(whole milk)	0.013968	0.126974	0.804028
0	(other vegetables)	(whole milk)	0.014837	0.121511	0.769430
2	(soda)	(whole milk)	0.011629	0.119752	0.758296

Visualization

Consistent with your practical formatting, we visualize the top 10 most purchased items to understand the dataset distribution.

```
# Plotting the most frequent items
top_items = df['itemDescription'].value_counts().head(10)

plt.figure(figsize=(10, 5))
top_items.plot(kind='bar', color='skyblue', edgecolor='black')
plt.title('Top 10 Most Purchased Items')
plt.xlabel('Item Description')
plt.ylabel('Total Count')
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Conclusion

In this practical, the Apriori Algorithm was successfully used to perform Market Basket Analysis.

- The data was transformed into a One-Hot encoded format suitable for association mining.
- We identified Frequent Itemsets using a support threshold, highlighting that "Whole Milk" and "Other Vegetables" are common staples.
- The Association Rules generated (filtered by Confidence and Lift) provide actionable insights into product relationships, demonstrating how this algorithm helps in retail strategy and recommendation systems.